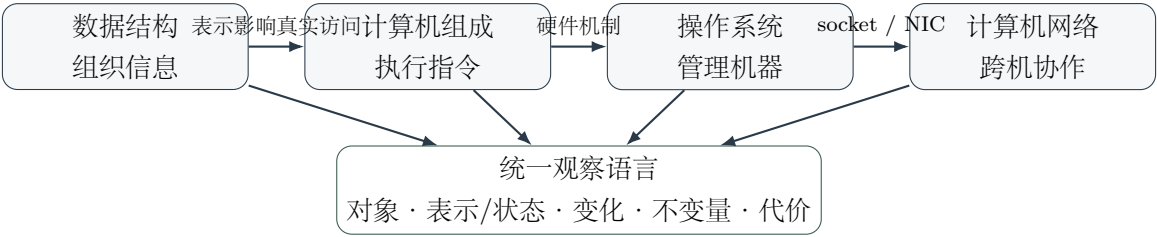


408 统一总图

四科心智模型 · 章节映射 · 做题入口

数据结构 · 计算机组成原理 · 操作系统 · 计算机网络



使用说明：这本总手册到底负责什么

总手册的唯一任务

本册不是四本教材的压缩版，也不是所有专题笔记的合集。它只负责回答四类问题：

1. 这门学科究竟研究什么世界？
2. 这门学科最少需要保留哪几个生成性母模型？
3. 一个知识点属于哪个专题，和相邻专题在哪里交接？
4. 面对陌生综合题，第一次应该从什么坐标进入推理？

总图 / 专题边界：四类手册各司其职

类型	任务	不应该做什么
总图 Atlas	建立母问题、母模型、专题地图、关键边界	不展开具体算法、协议步骤、寄存器级流程
专题 Topic	打穿一个核心机制，解释对象、状态、过程、不变量、计算模型	不重新建立整门学科总图
桥梁 Bridge	解释两个专题/学科的接口和状态交接	不重复两边全部知识
综合 Integration	在一个真实过程里同时调用多个已学机制	不成为新的知识大全

本册中的“完整”因此不是细节最多，而是**地图功能完整**：总定义、核心坐标、专题所有权、关键边界、跨科接口和做题入口全部齐备；详细机制与覆盖清单下沉到四科方法论及各专题手册。

目录

使用说明：这本总手册到底负责什么	ii
第一章 统一底座：四门课不是同一个模型，但可以用同一组问题观察	1
1.1 不要强行用一条公式统一四科	1
1.2 本册符号到底怎样读	1
1.3 四门课各自真正拥有的母模型	2
1.4 母模型怎样对应后续章节	3
1.5 四科在完整计算系统里的分工	4
1.6 跨四科反复出现的六种设计思想：必须落到实例	4
1.7 总手册统一做题入口：箭头是检查顺序	5
第二章 数据结构：为了操作而保存信息	7
2.1 学科母问题	7
2.1.1 逐格解释这条母链	7
2.2 一个例子把母模型完整跑一遍：为什么会有堆	7
2.3 四个生成性支点，以及它们在哪些章节出现	8
2.3.1 Relation 与 Representation 分离	8
2.3.2 Operation Set 决定结构价值	8
2.3.3 Invariant 是算法的稳定内核	8
2.3.4 Cost 不是单个 $O(\cdot)$ 标签	8
2.4 母模型一章节映射：学每一章时到底在深入哪一格	9
2.5 整门数据结构的专题地图	9
2.6 数据结构最重要的概念边界：不是背“≠”，而是学判据	10
2.7 数据结构做题入口：这条箭头怎样实际使用	11
2.8 一页压缩	11
第三章 计算机组成原理：软件语义怎样成为硬件时间过程	12
3.1 学科母问题	12
3.1.1 逐格解释	12
3.2 贯穿例子：一条 LOAD 怎样映射到整门计组	12

3.3	四个坐标轴，以及它们对应的章节	13
3.3.1	Semantic State: 什么对软件可见?	13
3.3.2	Data Location: 需要的数据在哪里?	13
3.3.3	Datapath / Control / Timing: 硬件如何在时间上组织工作?	13
3.3.4	Resource / Cost: 哪里会等待?	13
3.4	母模型—章节映射	13
3.5	计组专题地图	14
3.6	计组最重要的概念边界: 用题目信号判断	14
3.7	计组做题入口: 英文链怎样变成草稿动作	15
3.8	一页压缩	16
第四章	操作系统: 有限硬件怎样成为受保护的共享世界	17
4.1	学科母问题	17
4.1.1	这些词分别是什么意思	17
4.2	贯穿例子: 一次阻塞 read 怎样把模型跑起来	18
4.3	OS 的五个不可替代职责, 以及它们出现在哪里	18
4.4	OS 专题地图	19
4.5	母模型—专题映射: 每一章在改 S 的哪一部分	19
4.6	OS 需要牢牢记住的四条横向规律: 每条都要会落地	20
4.6.1	对象不是名字, 引用与映射决定关系	20
4.6.2	Blocked 与 Ready 之间隔着“事件完成”, Ready 与 Running 之间隔着“调度”	20
4.6.3	Mechanism 与 Policy 严格分开	20
4.6.4	Fast Path 失败时才进入 Kernel Slow Path	20
4.7	OS 最重要的概念边界: 判断题目到底在问哪一个状态	20
4.8	OS 做题入口: 九个英文词怎样变成实际推演	22
4.9	一页压缩	22
第五章	计算机网络: 没有全局即时状态时怎样协作	23
5.1	学科母问题	23
5.1.1	逐格解释	23
5.2	贯穿例子: TCP 超时重传怎样落进母模型	23
5.3	六个现实约束怎样生成网络机制	24
5.4	第一坐标: Scope——一个决定负责多远	24
5.5	第二坐标: State Ownership——谁保存完成决策所需的信息	25
5.6	三条贯穿网络的关系链: 每条都给出使用场景	25
5.6.1	Name → Address → Forwarding → Delivery	26
5.6.2	Reliability = Sequence + ACK + Timer + Retransmission + Window	26

5.6.3 Statistical Multiplexing → Queue → Congestion → Feedback	26
5.7 必须三分: Reliability / Flow / Congestion	26
5.8 Forwarding 与 Routing 必须彻底分开	26
5.9 母模型—章节映射	27
5.10 网络专题地图	27
5.11 一个网络请求的一生: 总图中的箭头怎样读	28
5.12 网络最重要的概念边界: 区别在哪里, 题目怎么用	28
5.13 网络做题入口: 八个词怎样变成动作	29
5.14 一页压缩	30
第六章 四科接口: 真正的系统不是四块并排知识	31
6.1 数据结构 × 计组: 渐近复杂度为什么不是实际机器时间	31
6.2 计组 × OS: 机制与策略的软硬件边界	31
6.3 OS × 网络: packet 最终必须变成进程可见的数据	31
6.4 一条“应用读取网络数据”的跨科路径	31
6.5 跨科综合题的统一顺序	32
第七章 专题所有权与学习路线: 总图怎样和后续手册配合	33
7.1 Canonical Owner 原则	33
7.2 推荐学习方式	33
7.3 什么时候应该修改总手册	34
第八章 覆盖导航: 总图只指出入口, 不复制清单	35
第九章 最后压缩: 考前真正应该留下什么	36

统一底座：四门课不是同一个模型，但可以用同一组问题观察

1.1 不要强行用一条公式统一四科

四门课确实共享“对象、状态、映射、不变量、成本”等思想，但它们研究的核心矛盾不同。真正稳妥的统一方式不是要求所有知识都套进同一个状态机，而是建立一个**观察镜头**：先用少量问题确定自己正在研究什么，再切换到该学科自己的母模型。

408 通用五问：这是观察顺序，不是万能公式

面对任何新概念，先问：

1. **Object / Goal**：现在研究的对象与最终目标是什么？

2. **Representation / State**：为了完成目标，系统保存了哪些信息？这些信息存在哪里？

3. **Operation / Event**：什么操作或事件会改变这些信息？

4. **Invariant / Boundary**：什么条件必须始终成立，哪些相似概念绝不能混用？

5. **Cost / Workload**：时间、空间、带宽、I/O、等待或可靠性代价来自哪里？

这里的五问不是要求每道题都机械写五行答案，而是防止一上来就套公式。例如做“TLB + 页表 + Cache”综合题时，第一问先确认对象是一次虚拟地址访问；第二问确认题目给出的状态包括 TLB、页表和 Cache；第三问确认事件是 CPU 发出一次访存；第四问区分 TLB miss、page fault、cache miss；第五问最后才计算访问次数或有效访问时间。换句话说，**先确定世界，再进入计算**。

1.2 本册符号到底怎样读

为了压缩复杂关系，本册会使用箭头和英文关键词。但它们只是一种**思考记号**，不是要求背诵的形式语言。第一次看到时按下面方式理解。

记号	在本册中的含义	怎样实际使用
----	---------	--------

$A \rightarrow B$	从 A 进入 B 的推理顺序、转换关系或依赖关系；具体含义由上下文决定，不等于“物理上 A 一定先发生”	例如 $Goal \rightarrow Representation$ 表示先知道目标，再检查表示； $Name \rightarrow IP$ 则表示名字经过解析得到网络地址
$A \xrightarrow{E} B$	当前状态 A 在事件/操作 E 发生后变成 B	例如 $Blocked \xrightarrow{I/O\ completion} Ready$ ；做状态题时必须能说出箭头上是什么事件
$A \neq B$	两个概念可能表面相似，但判定问题、状态所有者或作用域不同，不能互换	看到题目时要找“决定它属于 A 还是 B 的判据”，而不是只背一句不等式
$A + B + C$	三个因素需要共同存在或共同考虑，不是数值加法	如 $Seq+ACK+Timer$ 表示可靠传输至少同时依赖这些状态/机制
A/B	常用于并列两个观察维度或术语，表示“同时注意 A 与 B ”，不是除法	如 $Mechanism/Policy$ 表示分析时分别判断“能怎样做”和“选择怎样做”
$A \leftrightarrow B$	两侧存在接口、双向影响或反馈	如 $OS \leftrightarrow Network$ 表示 socket、buffer、NIC 等处存在状态交接

读箭头的固定方法

每看到一条母链，不要朗读英文单词。把每个框翻译成一句问题：“这一格让我现在看什么？”把每个箭头翻译成：“确认完前一格以后，下一步为什么有资格看后一格？”如果说不出这两句话，就说明这条模型还没有真正理解。

1.3 四门课各自真正拥有的母模型

数据结构：为操作需求选择可维护的表示

Problem → Relation → Operations → Representation → Invariant → Cost

这条链的中文意思是：先明确题目要解决什么，再看数据之间是什么关系；由需要频繁执行的操作决定应该怎样表示这些关系；算法每次修改表示时必须维护结构不变量；最后在具体 workload 下评价成本。

例如“需要不断插入元素并反复取当前最大值”：目标不是把所有元素完全排序，而是支持 $Insert + FindMax/DeleteMax$ ；因此只维护完全二叉树上的堆偏序即可，用数组保存； $FindMax$ 为常数时间，插入/删除极值通过上滤/下滤恢复 invariant。这个例子会分别在“堆”“数组表示”“复杂度”专题继续展开。

计算机组成原理：把 ISA 语义实现成真实硬件时序

ISA State → Data Location → Datapath/Control → Timing → Commit → Cost

中文意思是：先确定指令对软件承诺改变什么状态，再找操作数当前在哪里；然后追踪硬件经过哪些数据通路、由哪些控制信号驱动；判断值在什么周期产生和可用；最后确认何时对 ISA 可见并计算性能代价。

例如一条 `LOAD R1, 8(R2)`：软件语义是把地址 `R2 + 8` 处的数据写入 `R1`；硬件必须先读 `R2`、算有效地址、访问 `Cache/Memory`、等待数据返回，再在允许的时钟边界写回 `R1`。流水线、`Cache`、数据冒险和 `CPI` 都只是这条路径不同位置的问题。

操作系统：在事件中维护受保护的资源世界

Objects/Relations/Queues $\xrightarrow{\text{Event} + \text{Mechanism} + \text{Policy}}$ New State

中文意思是：先画清系统里有哪些对象、它们怎样引用/映射、谁正在排队等待；事件发生以后，硬件/内核用某种机制完成合法状态转换，策略决定多个合法选择中选哪一个；转换前后都必须满足保护、正确性和生命周期约束。

例如进程因 `read()` 等待磁盘：`Process` 是对象，等待队列记录“它在等谁”；`I/O completion` 是事件；`wakeup` 是机制，把它从 `Blocked` 变成 `Ready`；`scheduler policy` 之后才决定什么时候让它从 `Ready` 变成 `Running`。这样就不会把“唤醒”和“立刻运行”混成一件事。

计算机网络：没有全局即时状态时的分布式协作

Scope → Local State $\xrightarrow{\text{Message/Timer}}$ Transition → Feedback → Cost

中文意思是：先确定当前决策负责多远，再确定做决策的设备手里有哪些局部状态；收到报文或定时器到期后，它只依据这些本地信息更新状态并采取动作；动作通过新的报文影响其他节点；最后分析时延、带宽、队列和可靠性代价。

例如 `TCP` 超时重传：作用域是端到端；发送方保存未确认序号、定时器等本地状态；`timer expiration` 触发重传；新的 `segment/ACK` 构成反馈；性能上付出额外时延和带宽。这就是“局部状态 + 消息/定时器”的具体落地。

1.4 母模型怎样对应后续章节

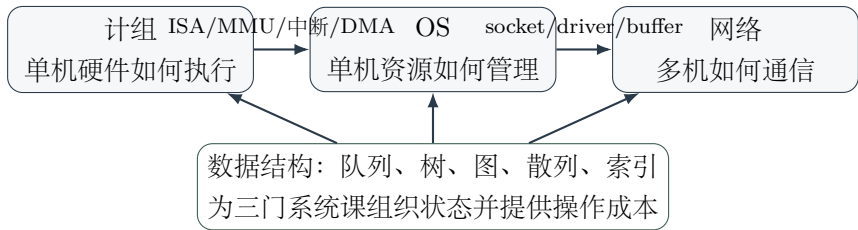
母模型不是替代章节目录，而是告诉你学习某一章时，到底在母模型的哪一格继续向下挖。

学科	母模型中的位置	典型章节	学习时真正要问
DS	Relation / Representation	线性表、树、图、散列、B+ 树	逻辑关系如何编码？保存了哪些额外信息？
DS	Invariant / Cost	排序、平衡树、图算法、复杂度	每一步怎样保持正确？为了快付出什么？

CO	Data Location / Path	寄存器、主存、Cache、数据通路	数据现在在哪，下一步经过谁？
CO	Timing / Commit	流水线、异常、中断、性能	值何时可用，何时算真正完成？
OS	Objects / Relations	进程、VM、文件系统	对象身份是什么，名字/映射怎样指向它？
OS	Queues / Event / Policy	调度、并发、I/O、页面置换	谁在等，什么事件改变资格，策略选谁？
NET	Scope / State Owner	分层、MAC/IP/Port、路由、TCP	当前决定负责多远，状态在端点/交换机/路由器谁手里？
NET	Message/Timer / Feedback	可靠传输、路由收敛、拥塞	收到什么或等多久以后改变状态，怎样影响别的节点？

1.5 四科在完整计算系统里的分工

数据结构与另外三科不是同一条硬件“层级”。更准确的关系是：**数据结构提供组织状态的通用工具**；计组、OS、网络分别赋予这些状态硬件、内核和分布式语义。



图中的箭头表示**知识接口**而不是“学习顺序”。例如数据结构中的 Queue 在 OS 里承载 Ready/Wait Queue，在网络里承载 packet buffer；队列的数据结构性质决定插入/取出成本，但“为什么谁应该先出队”属于 OS/网络自己的策略语义。

1.6 跨四科反复出现的六种设计思想：必须落到实例

总图保留六条横向规律，但每一条都必须能在具体章节找到实例，否则只是抽象口号。

设计思想	具体例子	为什么有用	对应专题
Representation / Indirection	Hash: Key→Bucket; OS: VA→PTE→Frame; Net: Name→IP	多一层映射换来更快定位、重定位、共享或抽象隔离	DS 散列; CO Cache; OS VM/FS; NET DNS/IP

State / Invariant	Heap 父子偏序；流水线最终 ISA 语义；PV 缓冲区容量；TCP 序号窗口	解释“为什么某一步能做/不能做”，是验证答案的依据	DS 算法；CO 流水线；OS 并发；NET 可靠传输
Scarcity / Sharing	ALU/总线冲突；CPU ready queue；共享链路 packet queue	需求超过瞬时供给时必然出现等待、排队或仲裁	CO hazard；OS 调度；NET MAC/拥塞
Locality / Caching	Cache line；TLB/Page Cache；DNS cache	用历史/邻近访问换取平均更快，但要处理 miss、替换和一致性	CO Cache；OS VM/IO；NET DNS
Fast / Slow Path	Hash 无冲突/冲突；Cache hit/miss；TLB hit/page fault；正常 ACK/timeout	常见情况走短路径，少见失败承担复杂修复成本	四科多个专题
Trade-off / Workload	Hash 点查快但不擅长范围；流水线吞吐高但有 hazard；RR 响应好但切换多；窗口大提高吞吐也增加在途状态	“最好”必须指定工作负载和成本目标	四科性能/算法选择题

1.7 总手册统一做题入口：箭头是检查顺序

总手册只负责**第一轮定位**，进入专题以后使用专题自己的解题协议。下面的箭头表示“前一项确认以后，再进入后一项”，不是说题目中的物理过程一定按这个顺序发生。

408 训练：跨科题第一次定位七问

Target → Scope/Topic → GivenState → Event/Operation

↓

Invariant/Boundary → Mechanism/Path → Cost

1. **Target**：题目最终要你求数值、判断状态，还是选择结构/机制？先把问号圈出来。

2. **Scope/Topic**：它属于哪门课、哪个专题、哪个层次？例如“下一跳”先进入网络转发，不要先想 TCP。

3. **GivenState**：题目已经给了哪些表、队列、寄存器、指针、窗口、Cache 行？先抄出真正会变化的状态。

4. **Event/Operation**：是谁做了什么，或者什么事件发生？没有事件依据，不要擅自改变状态。

5. **Invariant/Boundary**：有哪些规则限制答案？例如 heap 偏序、页内偏移不变、Ready/Blocked 不可混。

6. **Mechanism/Path**: 沿哪条表示、映射或数据通路执行? 需要时画表、时间线或状态图。
7. **Cost**: 最后才把路径转换成比较次数、cycle、I/O、时延、吞吐等数字。

小例子：为什么先定位再计算

题目问“某虚拟地址访问时 TLB 未命中，页表项有效，Cache 命中，主存访问几次？”先按入口链：Target= 访存次数；Topic= 地址翻译 +Cache；GivenState=TLB miss/PTE valid/Cache hit；Event= 一次 load；Boundary=TLB miss 不等于 page fault；Path= 查页表得到 PA，再访问 Cache；最后才根据题设的硬件访问模型计数。若第一步就把 TLB miss 当成 page fault，后面的数字算得再熟也会整题走错。

数据结构：为了操作而保存信息

2.1 学科母问题

数据结构真正研究什么

给定一组对象、它们之间的逻辑关系以及需要频繁完成的操作，应该保存哪些信息、采用什么表示，使操作在维持结构语义的同时具有可接受成本？

所以数据结构不是“容器目录”，而是：

Problem → Relation → Operations → Representation → Invariant → Cost

2.1.1 逐格解释这条母链

Problem 题目真正要解决的任务，例如“频繁找最大值”“判断连通”“支持范围查询”。

Relation 数据之间客观需要表达的关系，例如线性前后关系、树的层次关系、图的任意连接、集合归属关系。

Operations 使用者真正频繁执行的操作集合以及频率。它决定哪些能力值得提前维护。

Representation 计算机怎样把关系编码进数组位置、指针、矩阵、散列桶或索引节点。

Invariant 每次插入、删除、交换、旋转后仍必须成立的结构条件；它是判断算法是否正确的核心。

Cost 为这些能力付出的比较、移动、访存、空间、递归深度或外存 I/O。

箭头表示设计/解题时的因果顺序：不能脱离目标操作先宣布“用链表”或“用 Hash”；先有需求，才有表示选择。

2.2 一个例子把母模型完整跑一遍：为什么会有堆

假设任务是：数据不断到来，需要随时知道当前最大值，并经常删除最大值。

母模型位置	这个例子里具体是什么
Problem	动态维护一批元素，同时高频执行“取最大、插入、删除最大”。

Operations	$FindMax + Insert + DeleteMax$ ；并不要求任意元素之间完全有序。
Relation	只需要维护“父节点不小于孩子”的局部偏序，并利用完全二叉树保证紧凑。
Representation	完全二叉树可以直接用数组按下标表示父子位置，不必保存大量指针。
Invariant	对任意父子， $Parent \geq Child$ 。插入/删除破坏后用上滤/下滤恢复。
Cost	$FindMax = O(1)$ ，插入和删除最大值沿树高调整，约 $O(\log n)$ ；代价是不能像完全有序数组那样直接做有序遍历。

因此学习“堆”时，不应只背数组下标和上滤代码，而要看见：它是 **Operation Set 决定 Representation 与 Invariant** 的一个标准实例。

2.3 四个生成性支点，以及它们在哪些章节出现

2.3.1 Relation 与 Representation 分离

线性、层次、任意网络关系是逻辑结构；数组、链表、树节点、邻接矩阵/表是表示。同一逻辑关系可有多种表示，表示决定实际可见操作与成本。

实际章节：线性表里比较顺序表/链表；图里比较邻接矩阵/邻接表；树可以顺序或链式表示；外存索引则把“树”映射到磁盘页/块。

2.3.2 Operation Set 决定结构价值

堆、并查集、散列、B+ 树之所以不同，不是因为形状不同，而是它们主动支持的核心操作不同。问“哪个结构最好”之前必须先问 workload。

实际章节：Hash 强点查；B+ 树兼顾点查与范围并降低 I/O；并查集只保留集合代表关系；堆只维护极值所需偏序。

2.3.3 Invariant 是算法的稳定内核

BST 的有序性、Heap 的偏序、AVL 的平衡、并查集的代表元、图算法的已确定区域，都是算法真正维护的对象。代码只是实现不变量恢复的一种形式。

实际章节：快速排序的一轮 partition、Dijkstra 的已确定最短距离集合、Kruskal 的无环性、循环队列的空/满规则，都可以用“不变量”检查步骤是否合法。

2.3.4 Cost 不是单个 $O(\cdot)$ 标签

比较、移动、随机访问、额外空间、递归深度、cache locality、外存 I/O 都可能成为真正成本；当成本模型从 RAM 变成磁盘 I/O 时，最佳结构也会改变。

实际章节：内部排序主要关心比较/移动；外部排序关心 I/O 趟数；B/B+ 树关心树高和块访问；数组与链表即使渐近复杂度相同，机器局部性也可能不同。

2.4 母模型—章节映射：学每一章时到底在深入哪一格

章节/专题	主要对应母模型位置	学习时抓住的问题
复杂度	Cost	输入规模是什么？基本操作是什么？最好/平均/最坏/摊还口径是否一致？
线性表/数组	Relation → Representation	相同线性关系为什么连续表示和链式表示的定位/删除代价不同？
栈/队列	Operations → Semantics	为什么限制操作端点反而得到 LIFO/FIFO 的过程语义？
树/二叉树	Relation + Invariant	层次关系怎样带来递归结构，不同树靠什么 invariant 区分？
图遍历	State/Progress	Visited、Frontier、ExpansionOrder 如何共同推进探索？
图算法	Invariant	哪些结论已永久确定，局部松弛/选边为什么不会破坏全局正确性？
查找/散列/B+ 树	Operations → Representation → Cost	为了缩小候选集提前维护了什么信息，更新/空间/I/O 付什么代价？
排序	Operation + Invariant + Progress	每一趟之后哪个区域已经确定，算法如何逐步扩大正确区域？
KMP	Representation of history	怎样把已匹配历史压缩成 failure state，避免重复比较？

2.5 整门数据结构的专题地图

编号	专题	它真正回答的问题
DS-01	复杂度与成本模型	输入规模怎样变成基本操作次数，怎样区分最坏/平均/摊还/I/O 成本？
DS-02	线性表与数组映射	连续位置与显式指针分别换来什么能力与代价？
DS-03	栈、队列与受控遍历	限制操作顺序怎样制造 LIFO/FIFO 与 frontier 语义？
DS-04	树与递归层次	层次关系怎样缩小候选空间并支持递归分治？
DS-05	堆与并查集	只维护目标所需的最少信息，怎样支持极值与动态集合？
DS-06	图表示与遍历	任意关系中怎样维护 Visited、Frontier 与 Expansion Policy？

DS-07	图算法	MST、最短路、拓扑、关键路径分别维护什么“已确定”不变量？
DS-08	查找与有序索引	怎样用有序性、层次或索引减少候选集？
DS-09	散列	怎样把比较问题改造成 $\text{Key} \rightarrow \text{CandidateLocation}$ 与冲突管理？
DS-10	KMP	怎样保存已匹配信息避免失败后完全重来？
DS-11	内部排序	怎样通过局部动作不断扩大已确定有序区域？
DS-12	外部排序	当 I/O 成为主成本时，怎样减少归并趟数与随机访问？

2.6 数据结构最重要的概念边界：不是背“≠”，而是学判据

概念边界	为什么容易混 / 真正区别	题目中的判定信号	混淆后的典型错误
Logical Structure ≠ Storage Representation	一个描述“关系是什么”，一个描述“机器怎样保存关系”	题目说“线性表”还不能推断是数组还是链表；必须看顺序/链式存储	把“线性表”直接等同于连续内存，插删/地址计算全错
Structure ≠ Algorithm	结构是状态及 invariant，算法是改变状态的过程	问“AVL 是什么”与“插入后怎样旋转”分别属于对象定义和操作	背会代码但不知道何时需要修复、为什么修复
Heap Order ≠ Total Order	堆只保证父子偏序，不保证兄弟/跨子树全局大小	题目若问“第二大在哪里/能否二分查找”要警惕	误以为堆数组整体有序
Average $O(1)$ ≠ Worst-case $O(1)$	平均复杂度依赖分布/装填等假设，最坏界是对所有输入保证	Hash 题出现“平均”“最坏”“装填因子”“冲突”	把散列表任何情况下都写成常数时间
Shortest Path ≠ MST	一个优化“某些点之间的路径长度”，一个优化“连通全部顶点的总边权”	问源点到各点距离看最短路；问连通所有点且总成本最小看 MST	用 Prim/Kruskal 回答单源最短路
Stable Sort ≠ In-place Sort	稳定性讨论相等 key 的相对次序；原地性讨论额外空间	题目同时给“相同关键字记录”和“辅助空间”时要分别判断	把“稳定”误认为“不额外开数组”

Asymptotic Cost	渐近阶忽略常数和机器	外存、cache locality、	认为同为 $O(n)$ 的数
$\neq$ Actual Machine	层次，真实时间还受	数据规模很小时不能只	组/链表真实性能必然一
Cost	cache/I/O 等影响	看 $O(\cdot)$	样

2.7 数据结构做题入口：这条箭头怎样实际使用

408 训练：数据结构七步检查链

$Goal \rightarrow Relation \rightarrow Operations/Workload \rightarrow Representation$

↓

$Invariant \rightarrow State/Progress \rightarrow Cost/Boundary$

1. **Goal**：题目最后要求查找、更新、遍历、排序还是结构选择？

2. **Relation**：对象是线性、层次、图、集合还是 key→value 映射？

3. **Operations/Workload**：哪些操作出现得最多，是否有最坏界、稳定性、范围查询等额外要求？

4. **Representation**：题目实际用数组、指针、矩阵、邻接表、桶还是索引树？

5. **Invariant**：当前结构合法必须满足什么？把它写在草稿边上。

6. **State/Progress**：模拟算法时每一步哪些区域已经确定、frontier 在哪里、问题规模是否缩小？

7. **Cost/Boundary**：最后数比较/移动/空间/I/O，并检查空结构、首尾、重复 key、不连通等边界。

小例子：题目问“频繁插入并删除最大值，选什么结构？”

不要看到“最大值”就直接答排序。按七步走：Goal= 动态维护；Relation= 元素集合本身没有必要完全有序；Operations=Insert/DeleteMax 高频；Representation 候选有有序数组、BST、Heap；Invariant 只需保证能快速找到极值；Cost 比较后，Heap 用较弱的父子偏序换取 $FindMax = O(1)$ 与更新 $O(\log n)$ 。这就是从需求推出结构，而不是从关键词猜结构。

2.8 一页压缩

闭眼后应该剩下什么

$Problem \rightarrow Relation \rightarrow Operations \rightarrow Representation \rightarrow Invariant \rightarrow Cost$

快速查询几乎总来自提前维护更多信息；保存越多不是越好，只保存足以完成目标的关系；算法的本质是在局部修改后恢复/保持结构不变量。看到箭头时，把它读成“下一步我要问什么”，而不是背一串英文。

计算机组成原理：软件语义怎样成为硬件时间过程

3.1 学科母问题

计组真正研究什么

如何用有限数量、有限速度、有限带宽的真实硬件，正确而高效地实现 ISA 对软件承诺的程序语义？

一条指令从软件看可以抽象为状态转移 $S_{t+1} = F_I(S_t)$ 。这里 S_t 指程序员可见的体系结构状态，例如 PC、通用寄存器和可见内存； F_I 表示当前指令 I 规定的语义。计组真正要解释的是：这个看起来“一步完成”的语义，怎样在硬件内部被拆成若干数据移动和时间阶段。

母模型：

ISA State $\rightarrow$ Data Location $\rightarrow$ Datapath/Control $\rightarrow$ Timing $\rightarrow$ Commit $\rightarrow$ Cost

3.1.1 逐格解释

ISA State 指令执行前后，软件能够观察到哪些状态变化；这是硬件必须兑现的承诺。

Data Location 本次运算所需指令和操作数当前在寄存器、Cache、主存还是设备中。

Datapath / Control 数据实际经过 ALU、MUX、寄存器、总线、Memory interface 的路径；控制信号决定本周期哪些路径被启用。

Timing 每个值在哪个周期产生、什么时候稳定、什么时候能被后续部件使用。

Commit 内部可能预测、并行、暂存，但最终什么时候才能把结果作为“这条指令已经完成”的体系结构结果暴露。

Cost 关键路径、cycle、CPI、stall、miss penalty、带宽等性能代价。

箭头表示分析顺序：先知道指令应该做什么，才能判断数据该去哪；只有路径明确以后，才能正确谈时序和性能。

3.2 贯穿例子：一条 LOAD 怎样映射到整门计组

以 LOAD R1, 8(R2) 为例。这里不展开具体 ISA 编码，只用它导航整门学科。

母模型位置	LOAD 中实际发生什么
ISA State	语义目标： $R1 \leftarrow M[R2 + 8]$ ，并按 ISA 规则推进 PC。
Data Location	指令要先被取到； $R2$ 在寄存器；目标数据可能在 L1/L2/DRAM。
Datapath / Control	读寄存器 $R2$ ，ALU 计算有效地址，选择数据存储端口，返回数据后选择写回 $R1$ 。
Timing	地址在哪一级产生？ Cache hit 数据何时可用？ 若后续指令马上读 $R1$ ，需要 forwarding 还是 stall？
Commit	只有符合微架构/ISA 提交规则后， $R1$ 的新值才成为软件可见结果。
Cost	Cache miss、结构冲突、数据依赖都会增加等待；性能不能只由时钟频率决定。

于是：指令系统讲“LOAD 承诺什么”；数据通路讲“怎么走”；流水线讲“什么时候走”；Cache 讲“数据在哪里”；性能讲“等了多久”。这就是总图中“一个母例题串全科”的意义。

3.3 四个坐标轴，以及它们对应的章节

3.3.1 Semantic State：什么对软件可见？

对应：数据表示、ISA、指令系统、异常/中断的体系结构语义。学习时间：同一串 bit 在当前语义下代表什么？这条指令最终允许改变哪些可见状态？

3.3.2 Data Location：需要的数据在哪里？

对应：寄存器、主存、Cache、I/O。学习时间：目标数据当前在哪一级？如果不在，下一层是谁？搬运成本是什么？

3.3.3 Datapath / Control / Timing：硬件如何在时间上组织工作？

对应：单周期、多周期、流水线、硬布线/微程序控制。Datapath 决定有哪些路，Control 决定此刻打开哪条路，Clock 决定状态何时采样。

3.3.4 Resource / Cost：哪里会等待？

对应：hazard、总线仲裁、Cache miss、I/O、性能计算。ALU、端口、总线和存储系统都有限；“值没准备好”和“资源被占用”都会变成 stall。

3.4 母模型—章节映射

章节/专题	主要对应母模型位置	学习时真正要问
数据表示与运算	ISA State / Semantics	有限 bit 如何解释，溢出/舍入条件是什么？
指令系统/寻址	ISA State → Data Location	指令承诺什么，操作数如何被定位？
主存组织	Data Location / Resource	地址线、数据线、bank 怎样决定容量与并行访问？
数据通路与控制	Datapath / Control	一条指令被拆成哪些微操作，哪些控制信号驱动？
流水线	Timing / Availability	每个值在哪一级产生，hazard 为什么迫使等待/转发/预测？
Cache	Data Location / Fast-Slow Path	目标块放哪、如何识别、miss 后去哪、写策略怎样维护副本？
总线与 I/O	Resource / Interface	多部件如何共享通信资源，异步设备怎样通知/搬运？
性能	Cost / Bottleneck	IC、CPI、周期时间、miss/stall 怎样组合成完成时间？

3.5 计组专题地图

编号	专题	核心问题
CO-01	数据表示与运算	有限位宽怎样表达整数、浮点与运算语义，溢出/舍入从哪里来？
CO-02	ISA 与寻址方式	软件可见机器承诺是什么，一条指令怎样描述操作与操作数？
CO-03	主存与芯片组织	容量、位宽、bank 和地址线怎样落成实际存储系统？
CO-04	数据通路与控制	指令如何被分解成微操作，控制信号如何驱动数据移动？
CO-05	单周期、多周期与流水线	工作怎样沿时间切分与重叠，hazard 为什么出现？
CO-06	Cache 与存储层次	如何利用局部性管理副本，placement/tag/replacement/write 各解决什么？
CO-07	总线与 I/O	同步 CPU 如何与异步设备通信，Polling/Interrupt/DMA 分别卸载什么？
CO-08	性能与并行	IC、CPI、周期时间、瓶颈、并行度怎样共同决定完成时间？

3.6 计组最重要的概念边界：用题目信号判断

概念边界	真正区别	题目信号	混淆后的错误
ISA ≠ Microarchitecture	ISA 是软件可见承诺；微架构是内部实现方法	问“指令/寄存器/寻址语义”看 ISA；问“流水级/预测/内部端口”看实现	把某 CPU 的实现细节当成所有同 ISA 机器都必须如此
CPI 低 ≠ 程序必然快	总时间还由 IC 和 cycle time 决定	比较机器时题目常同时给 IC/CPI/频率	只比较 CPI 得出性能结论
Pipeline Throughput ≠ Instruction Latency	流水线主要提高单位时间完成指令数，不代表单条指令经过的级数减少	问“填满后每周期完成几条”与“某条从 IF 到 WB 多久”是不同量	把五级流水说成每条指令只需 1 cycle
Address Field ≠ Full Address	指令字段可能是寄存器号、立即数、偏移，需要寻址方式计算有效地址	看到 base+offset、间接寻址、PC-relative	直接把指令中的字段当绝对地址
Cache Miss ≠ Page Fault	一个是硬件存储层次副本未命中；一个是虚拟内存映射/驻留异常，进入 OS	题目分别给 tag/index 与 PTE/present 位	Cache miss 后错误地直接认为要访问磁盘/陷入 OS
Interrupt ≠ DMA	Interrupt 解决“完成后怎样通知 CPU”；DMA 解决“数据怎样批量搬运”	题目问 CPU 是否逐字节搬、谁产生完成通知	把 DMA 理解成不需要中断/CPU 完全不参与
Clock Rate 高 ≠ Workload 更快	高频率可能伴随更高 CPI、更多指令或不同 miss 行为	真正比较同一程序完成时间时必须综合指标	只看 GHz 判断处理器快慢

3.7 计组做题入口：英文链怎样变成草稿动作

408 训练：计组六步检查链

State → DataLocation → Path → Resource → Timing → Commit/Cost

1. State：题目执行前 PC、寄存器、Cache/Memory 关键状态是什么？目标指令最终应该改变什么？

2. DataLocation：指令和操作数当前在哪一级；若 miss，下一层在哪里？

3. Path：画最小数据通路，只标当前题真正经过的部件和 MUX。

4. Resource：同一周期是否有人争 ALU、端口、总线或存储器？值是否还没准备好？

5. **Timing**: 逐 cycle 标值何时产生、何时可用, 必要时画流水表。
6. **Commit/Cost**: 确认最终可见状态, 然后再算 CPI、时间、命中/未命中代价。

小例子: LOAD 后紧跟 ADD 使用 R1

先别背“数据冒险要停几拍”。State: LOAD 要写 R1; DataLocation: 数据要到 MEM/Cache 后才产生; Path: LOAD 经过 EX 算地址、MEM 取数据, ADD 在自己的 EX 级需要 R1; Resource/Timing: 比较“数据产生时刻”和“ADD 需要时刻”; 若 forwarding 仍来不及才 stall。这样停几拍是从时间关系推出来的, 而不是死背固定数字。

3.8 一页压缩

闭眼后应该剩下什么

ISA State → Data Location → Datapath/Control → Timing → Commit → Cost

硬件可以内部重叠、缓存、预测和复用, 但不能改变 ISA 语义; 性能问题先追踪数据位置、可用时刻和等待来源, 再谈 CPI 或频率。

操作系统：有限硬件怎样成为受保护的共享世界

4.1 学科母问题

OS 真正研究什么

当多个彼此独立甚至互不信任的程序共享 CPU、内存、设备和持久存储时，内核怎样建立抽象、保持控制、协调等待与竞争、隔离权限，并在事件和故障中维持合法状态？

OS 的最简状态模型：

$$S = (Objects, Relations, Queues).$$

统一系统推理引擎（OS System Reasoning Engine）：

State $\rightarrow$ Transition $\rightarrow$ Failure (Bad State) $\rightarrow$ Property (Safety / Liveness) $\rightarrow$ Minimal Mechanism $\rightarrow$ Tradeoff

当事件触发状态转移：

$$S_t \xrightarrow{Event+Mechanism+Policy} S_{t+1},$$

必须维持物理与逻辑不变量 $I(S_{t+1}) = \text{true}$ 成立，同时优化 Cost。

4.1.1 这些词分别是什么意思

Objects 内核正在管理的“东西”，例如 Process、Thread、AddressSpace、Page、File、OpenFile、DeviceRequest。

Relations 对象之间怎样互相引用或映射，例如 $fd \rightarrow \text{OpenFile} \rightarrow \text{inode}$ ， $VPN \rightarrow \text{PTE} \rightarrow \text{PFN}$ ，父进程 $\rightarrow$ 子进程。

Queues 当前不能继续的请求/实体在哪里等待，例如 ready queue、I/O wait queue、lock waiters、device request queue。

Event 真正触发变化的事情，例如 timer interrupt、page fault、I/O completion、unlock、system call。

Mechanism 内核/硬件“有能力怎样做”，例如 context switch、trap entry、wakeup、page-table update。

Policy 当存在多个合法选择时“应该选谁/什么时候做”，例如 scheduler 选谁、replacement 牺牲哪页、I/O scheduler 排哪个请求。

Invariant 任何合法状态都必须满足的规则，例如权限不能越界、互斥不能同时被两者占用、页表映射必须符合保护位。

Cost 等待、切换、TLB/cache miss、I/O、竞争、写回等代价。

箭头 $S_t \xrightarrow{E} S_{t+1}$ 的最重要含义是：没有事件，不要凭感觉改状态；有事件，还要说清是谁用什么机制改了哪一部分状态。

4.2 贯穿例子：一次阻塞 read 怎样把模型跑起来

假设进程 A 调用 `read()`，目标数据不在页缓存，需要磁盘读取。

模型位置	这个例子里具体是什么
Objects	Process A、fd/OpenFile/inode、page-cache page、I/O request、device。
Relations	A 的 fd 指向 OpenFile，OpenFile 指向文件对象；文件偏移映射到目标数据页/块。
Queues	A 因等待数据进入 wait queue；磁盘请求进入设备请求队列；其他进程仍可能在 ready queue。
Event	<code>read()</code> 系统调用、cache miss、I/O completion interrupt。
Mechanism	trap 进入内核、提交 I/O、把 A 置 Blocked、completion 时 wakeup 成 Ready。
Policy	A 阻塞后 scheduler 选哪个 Ready 进程；A 被唤醒后何时再次得到 CPU。
Invariant	Blocked 进程不能被当成 Ready；权限/文件引用必须合法；I/O 数据完成后才能向用户返回。
Cost	系统调用、context switch、设备 I/O 和等待时间远高于纯内存 fast path。

这个例子分别连接进程/调度、文件系统、Page Cache、I/O、中断；总图只负责告诉你这些专题在同一条状态轨迹里在哪里交接，具体细节由专题册拥有。

4.3 OS 的五个不可替代职责，以及它们出现在哪里

职责	含义	典型章节/例子
Control	用户代码可以直接高效执行，但内核必须能受控进入/收回 CPU	user/kernel mode、system call、trap、timer interrupt
Virtualization	把有限复杂硬件包装成稳定私有/逻辑抽象	process/thread、virtual address space、file/fd
Coordination	多个执行流争 CPU、锁、buffer、device 时需要排队与唤醒	scheduling、PV/lock、I/O wait、device queue

Protection	规定“谁能对哪个对象做什么” 并让硬件可执行地检查	privilege、page permissions、file permissions、 isolation
Persistence	状态跨运行时存在，多步更新 失败后仍要有可恢复语义	file system、writeback、fsync、journaling/crash consistency

4.4 OS 专题地图

编号	专题	核心问题
OS-01	控制权、系统调用与中断异常	用户代码怎样安全进入/离开内核，mode switch 与 context switch 如何区分？
OS-02	进程、线程与调度	执行实体如何创建、阻塞、唤醒、被调度并最终退出？
OS-03	并发、锁与 PV	共享状态面对任意交错时怎样保持 safety、ordering 与 progress？
OS-04	虚拟内存与地址翻译	VA 怎样映射到 PA，fault、TLB、COW、replacement 各解决什么？
OS-05	I/O 系统	快 CPU 怎样接入慢而异步的设备，Interrupt、DMA、Buffer/Cache 各自作用是什么？
OS-06	文件系统	Name、fd、open state、inode 与 block 怎样解耦并形成可命名持久对象？
OS-07	持久化与崩溃一致性	多步写入中途失败时怎样保证或恢复合法持久状态？
OS-B1	CPU × OS 桥梁	特权级、MMU、TLB、中断、Timer、DMA 的软硬件分工边界是什么？
OS-II	OS 综合状态机	read/mmap/fork/COW/pipe/memory pressure 等真实过程怎样跨多个专题运行？

4.5 母模型—专题映射：每一章在改 S 的哪一部分

专题	主要改变/维护的状态	用母模型观察时间什么
进程/调度	进程对象 + Ready/Blocked 等队列	什么事件改变资格？wakeup 与 schedule 谁负责？
并发/PV	共享对象 + 锁/信号量状态 + 等待队列	哪个 invariant 被交错威胁？谁持有、谁等待、谁能制造条件？

虚拟内存	VA→PTE→frame relations + residency state	这次访问是 translation、permission 还是 residency 问题?
I/O	I/O 请求 + 设备队列 + 完成事件	请求提交后谁等? 数据谁搬? 完成事件怎样把进程变 Ready?
文件系统	Name、fd、OpenFile、inode 与 block 的引用/映射关系	名字、打开实例、文件实体、数据位置分别是哪一层对象?
持久化	内存状态 ↔ 稳定介质状态	write 返回到哪一步? crash point 后哪些状态允许存在?

4.6 OS 需要牢牢记住的四条横向规律：每条都要会落地

4.6.1 对象不是名字，引用与映射决定关系

PID、fd、虚拟地址、路径名都不是对象本身。例如两个 fd 可以指向同一个 OpenFile，也可以通过两个独立 OpenFile 最终指向同一 inode。题目一旦问“共享 offset 吗”，就不能只看文件名相同，而要追踪引用链。

4.6.2 Blocked 与 Ready 之间隔着“事件完成”，Ready 与 Running 之间隔着“调度”

磁盘完成会把等待进程从 Blocked 变 Ready；它只获得竞争 CPU 的资格。只有 scheduler 选中后才 Ready→Running。做 I/O/调度综合题时，这两根箭头必须分开画。

4.6.3 Mechanism 与 Policy 严格分开

Context switch 是“能切换”的机制，scheduler 选谁是策略；page fault handler 能回收/建立映射是机制，replacement 选 victim 是策略。题目问“由谁决定下一个进程”时，不要回答“上下文切换”。

4.6.4 Fast Path 失败时才进入 Kernel Slow Path

TLB hit、page-cache hit、无竞争锁等常见情况尽量便宜；fault、I/O、锁竞争、回收与恢复进入慢路径。性能题要先判断“这次到底命中了哪条路径”，再累加代价。

4.7 OS 最重要的概念边界：判断题目到底在问哪一个状态

概念边界	真正区别	题目信号	混淆后的错误
------	------	------	--------

Process $\neq$ Program	Program 是静态代码/数据；Process 是运行中的执行与资源状态	看到 PID、状态、地址空间、调度就是 process	认为同一程序只能有一个进程
Ready $\neq$ Blocked	Ready 只缺 CPU；Blocked 还缺某个外部条件/资源	“时间片到”通常 Running $\rightarrow$ Ready；“等待 I/O/信号量”是 Blocked	I/O 完成后直接写 Running，跳过调度
Mode Switch $\neq$ Context Switch	mode switch 可仍是同一执行实体；context switch 改变 CPU 当前执行实体	system call/interrupt 不一定换进程；schedule 才涉及执行实体切换	认为每次系统调用都一定切换进程
Mutex $\neq$ Condition	Mutex 保护谁能同时进入；Condition 表达“什么时候才允许继续”	看到共享临界区看互斥；看到“必须等到 buffer 非空”看条件同步	用一把锁代替状态条件，或持锁等待造成死锁
Signal/Wakeup $\neq$ Predicate True	通知只是“状态可能变化”；真正条件仍要重新检查	condition variable 常用 while(predicate false) wait	被唤醒后不检查条件直接继续
TLB Miss / Page Fault / Cache Miss	翻译缓存、虚拟页映射/驻留、数据缓存是三个不同层次状态	看 TLB / PTE / present / cache tag 各自给了什么	一看到 miss 就默认磁盘 I/O
Buffer $\neq$ Cache $\neq$ Spooling	Buffer 吸收速率/粒度差；Cache 为复用保存副本；Spooling 用队列虚拟化独占设备	题目问“平滑生产消费”“重复访问”“打印排队”分别不同	把任何内核内存都叫缓存，机制目的判断错
File Name $\neq$ inode $\neq$ Open File State	Name 属命名空间；inode 是文件实体元数据；OpenFile 是一次打开的运行状态（如 offset）	hard link/unlink 看 name $\leftrightarrow$ inode；dup/fork offset 看 OpenFile	认为删除名字就立即销毁对象，或两次 open 必共享 offset
write() return / Durable	返回通常表示数据已被内核接受，不等于稳定介质已经持久化	题目出现 page cache / dirty / fsync / crash	把系统调用返回时刻当作物理落盘时刻

4.8 OS 做题入口：九个英文词怎样变成实际推演

408 训练：OS 九步检查链

Resource/Abstraction → Objects/Relations/Queues → Event → Boundary

↓

Mechanism → Policy → Invariant → Wait/Fault/Recovery → Cost

1. **Resource/Abstraction**：底层真实资源是什么，上层看到的抽象是什么？
2. **Objects/Relations/Queues**：画出题目真正涉及的对象、引用/映射和等待队列。
3. **Event**：逐步写“谁发生了什么”，不要无事件改变状态。
4. **Boundary**：当前是 user/kernel、hardware/kernel、name/object、Ready/Blocked 哪个边界？
5. **Mechanism**：系统具备用什么机制完成变化？trap、switch、wakeup、PTE update、DMA？
6. **Policy**：如果有多个候选，策略选谁？没有选择问题就不要硬找 policy。
7. **Invariant**：权限、互斥、容量、引用生命周期、映射合法性有哪些必须保持？
8. **Wait/Fault/Recovery**：正常路径失败后，是等待、fault、回收还是恢复？
9. **Cost**：最后计算等待、切换、访存/I/O 次数等代价。

小例子：I/O 完成后进程是什么状态？

题目说 A 因读磁盘阻塞，之后磁盘中断到达。Objects/Queues：A 在 I/O wait queue；Event=I/O completion；Mechanism=wakeup，把 A 移入 ready queue；Policy 只有在调度时才决定是否选 A；Invariant=Blocked/Ready 语义不能混。因此答案首先是 Ready，而不是凭“数据到了”直接写 Running。这个判断正是九步链在一道选择题里的缩写版本。

4.9 一页压缩

闭眼后应该剩下什么

Objects + Relations + Queues $\xrightarrow{\text{Event} + \text{Mechanism} + \text{Policy}}$ NewState

做题时把这句话翻译成：谁是什么对象？它和谁什么关系？谁在等？发生了什么？系统靠什么改变状态？如果有多个合法选择由什么策略选？改变后是否仍合法？这比背九个英文词更重要。

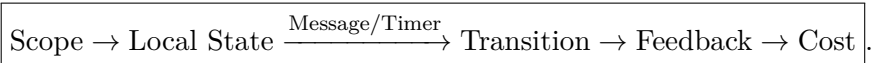
计算机网络：没有全局即时状态时怎样协作

5.1 学科母问题

网络真正研究什么

大量自治节点只能看到局部状态，信息传播有时延、链路可能不可靠、资源还要共享时，怎样仅靠消息交换共同形成可用的端到端通信？

网络统一模型：



5.1.1 逐格解释

- Scope** 当前决策负责多远：一个应用语义、两个端点、跨多个网络、当前一跳，还是一段物理介质？
- Local State** 做决定的设备现在真正知道什么；网络中没有一个节点可以瞬时读取所有其他节点状态。
- Message / Timer** 网络状态机最常见两类输入：收到消息，或者“等了足够久仍没等到消息”。
- Transition** 设备依据本地状态和输入更新表项、窗口、序号、队列或连接状态，并决定发送/丢弃/重传等动作。
- Feedback** 本节点的动作通过新报文影响别的节点，使分布式系统逐步协调。
- Cost** 时延、带宽、队列、丢包、在途数据量、重传和控制开销。

箭头表示分析一个网络机制时的观察顺序，不是说真实 packet 一定按这六个词逐层“走”。例如 Scope 是分析者先确定的坐标，不是网络中真的存在一个叫 Scope 的协议层。

5.2 贯穿例子：TCP 超时重传怎样落进母模型

母模型位置	TCP 超时场景
Scope	端到端：发送方和接收方之间的一条 TCP 连接。

Local State	发送方保存已发送未确认的序号范围、重传计时器、窗口等；路由器通常不保存这些 TCP 序号状态。
Message/Timer	正常时 ACK 到达；异常时 timer expiration 表示发送方在预期时间内没有获得足够反馈。
Transition	发送方更新重传/拥塞相关状态并重新发送相应 segment。
Feedback	重传数据到达接收方，接收方 ACK 又返回发送方，继续改变其状态。
Cost	超时增加至少一个等待周期，并消耗额外带宽；窗口/拥塞策略还会影响后续吞吐。

这就是“网络是耦合的局部状态机”最小实例。学习 GBN、SR、TCP、路由收敛和拥塞时，都应该尝试找出各自的 Local State、Message/Timer 和 Feedback。

5.3 六个现实约束怎样生成网络机制

约束	直接困难	机制方向	在哪些章节看到
Distance	远端状态不能瞬间知道，传播要时间	RTT、Timer、Pipeline、Window	性能、可靠传输、TCP
Unreliability	bit/packet 可能错、丢、重、乱	CRC/Checksum、Seq、ACK、Retransmission	链路差错、可靠传输
Sharing	多个流共享同一介质/链路/队列	MAC、Multiplexing、Queue、Congestion	LAN/MAC、拥塞
Heterogeneity	不同介质、设备与局域网技术不统一	Layering、统一网络层抽象、IP	体系结构、IP
Scale	全球节点太多，不能保存逐主机平坦状态	Prefix、Hierarchy、Aggregation、AS	CIDR、路由、BGP
No Global Instantaneous State	每个节点只能用延迟到达的局部信息推断	Distributed Routing、Timeout、Feedback、Convergence	路由、TCP、拥塞

这张表的使用方式是：学到一个新机制时倒着问“如果没有左边那个现实约束，这个机制是否必要？”例如没有距离和 RTT，Stop-and-Wait 的低利用率问题就会弱很多；没有共享资源，拥塞控制就失去核心动机。

5.4 第一坐标：Scope——一个决定负责多远

作用域	核心问题	典型对象/状态与题目例子
Application	双方交换的语义是什么？	HTTP method/status、DNS query/answer；问“请求含义/应用交互”在这里
Transport	到哪个端点、需要什么端到端通信条件？	Port、connection、Seq/ACK、window；问“进程、可靠、流控”在这里
Network	跨网络目的地在哪里，当前节点下一跳去哪？	IP、prefix、forwarding state；问“子网、LPM、路由”在这里
Link	当前这一跳怎样交给下一设备？	Frame、MAC、CRC、switch state；问“交换机、CSMA、帧”在这里
Physical	bit 怎样成为介质中的信号？	Symbol、encoding、modulation、bandwidth；问Nyquist/Shannon/码元在这里

纵向通信主链：

Meaning → Endpoint → Path → One Hop → Signal

这条箭头的意思是：一个应用消息要真正传出去，必须逐步补齐“端点是谁、跨网目的地是谁、这一跳交给谁、最后怎样编码成信号”的信息。封装就是为不同作用域加入完成局部决策所需的控制信息，而不是为了背“每层加首部”。

5.5 第二坐标：State Ownership——谁保存完成决策所需的信息

- TCP connection/Seq/ACK/window：主要属于通信端点；
- switch：维护 MAC→Port；
- router data plane：维护 Prefix→NextHop/Interface；
- routing protocol：维护形成/更新上述转发表所需的控制状态；
- DNS resolver/server：维护 Name→ResourceRecord 与 TTL/authority 相关状态。

“谁保存状态”怎样用于题目

如果题目问“某 TCP ACK 到达后谁更新发送窗口”，先问状态所有者：TCP endpoint。普通中间路由器只需处理 IP forwarding，不会因为看到了 TCP ACK 就维护该连接的 seq / window。反过来，题目问“目的 IP 最长前缀匹配哪一项”，状态所有者是当前路由器的 forwarding table，不要跑去分析发送方 TCP 状态。

5.6 三条贯穿网络的关系链：每条都给出使用场景

5.6.1 Name → Address → Forwarding → Delivery

Name → IP Address → Forwarding Decision → NextHop → Link Address .

中文意思：用户/应用先用名字描述“我要找谁”；DNS 等机制得到网络层地址；当前节点用目的 IP 和转发表决定下一跳；进入具体链路后，还要得到下一跳的链路层地址。做“浏览器访问网站”“ARP 与路由器”“同网段/异网段”综合题时，用这条链逐层追踪。

5.6.2 Reliability = Sequence + ACK + Timer + Retransmission + Window

“+”表示可靠传输需要这些信息/机制共同配合。Sequence 用于区分新旧/顺序，ACK 提供成功反馈，Timer 处理“没有反馈”的情况，Retransmission 修复丢失，Window 允许多个数据在途提高利用率。链路层 GBN/SR 与 TCP 都会调用这套模型，但作用域和具体规则不同。

5.6.3 Statistical Multiplexing → Queue → Congestion → Feedback

多个流共享资源提高平均利用率；突发到达超过服务能力时，请求先排队；持续过载使延迟/丢包上升形成拥塞；端点只能通过丢包、RTT、显式信号等反馈调节负载。做拥塞题时不要从 cwnd 公式开始，而要先找“哪个有限资源正在排队”。

5.7 必须三分：Reliability / Flow / Congestion

机制	保护对象	做题时问什么	典型信号
Reliability	通信语义	数据有没有正确、完整地到达？	seq、ACK、timeout、duplicate、retransmit
Flow Control	Receiver	接收端缓冲和处理能力能不能承受？	receive window、receiver buffer、rwnd
Congestion Control	Network	中间路径共享资源能不能承受？	cwnd、loss/RTT、slow start、AIMD、queue

三者都可能让“发送量变小”，所以容易混；真正区分标准不是动作，而是它在保护谁的状态。

5.8 Forwarding 与 Routing 必须彻底分开

Data Plane / Control Plane：一个用表，一个造表

Forwarding：当前 packet 已到某台路由器，使用现有 forwarding table 做局部快速决策。

$DestinationIP \xrightarrow{LPM} NextHop.$

这里箭头表示“输入目的地址，通过最长前缀匹配查表，输出下一跳”。

Routing：多个路由器怎样交换控制信息，让 forwarding table 被建立或更新。

$$LocalKnowledge + RoutingMessages \rightarrow UpdatedRoutingState.$$

题目若给一张路由表让你查下一跳，是 forwarding；题目若让你迭代 DV、运行 Dijkstra、分析 RIP/OSPF/BGP，是 routing/control plane。

5.9 母模型—章节映射

章节/专题	主要对应母模型位置	学习时真正要问
物理/性能	Cost + Distance	一 bit/packet 的发送、传播、排队和吞吐代价怎么形成？
链路/MAC	One-hop Scope + Local State	同一介质上谁能发、帧怎样一跳交付、错误怎样检测？
可靠传输	Message/Timer + Feedback	数据/ACK 丢失或延迟时，端点状态机怎样修复？
IP/转发	Network Scope + Forwarding State	目的地址如何被 prefix match 映射成 next hop？
路由	Distributed Local State + Feedback	每个路由器只靠局部消息怎样逐步形成可用路径知识？
TCP/UDP	Endpoint Scope + Connection State	主机间 IP 服务怎样变成进程间通信与连接状态？
拥塞	Queue + Feedback + Cost	需求超过路径能力时，端点怎样从有限反馈调整负载？
DNS/HTTP 等应用	Meaning / Name	名字怎样变地址，bytes 怎样获得应用语义？

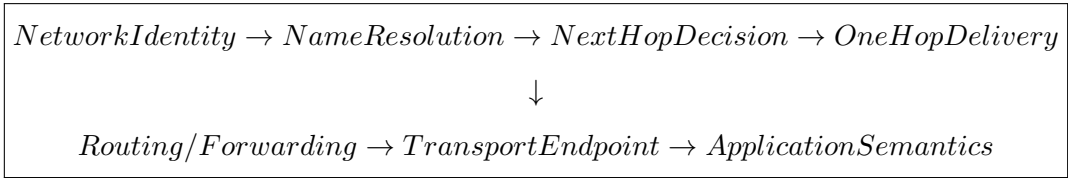
5.10 网络专题地图

编号	专题	唯一母问题
NW-01	通信基础与网络性能	一个 bit 穿过真实信道要付什么时间和容量代价？
NW-02	单跳交付：帧、MAC、LAN	下一设备已知后，这一跳怎样共享介质、检测错误并完成交付？

NW-03	可靠传输	怎样用 Seq/ACK/Timer/Retransmission/Window 在不可靠信道上构造可靠服务？
NW-04	IP 地址、子网与分组转发	目的地址怎样通过 Prefix Match 变成下一跳？
NW-05	路由与控制平面	转发表从哪里来，局部知识怎样通过消息逐步收敛？
NW-06	运输层与 TCP 状态机	IP 到主机后怎样交给进程，并建立端点之间的连接状态？
NW-07	拥塞与反馈控制	Demand>Capacity 时怎样通过反馈调节在途负载？
NW-08	应用层：发现与语义	底层能传 bytes 以后，怎样解决 Name→Address 与 Bytes→Meaning？
NW-I1	一个网络请求的一生	DHCP/DNS/ARP/IP/TCP/HTTP 等机制在真实访问中怎样接起来？
NW-B1	OS × Network 桥梁	socket、协议栈、buffer、NIC、DMA、中断和进程唤醒怎样交接？

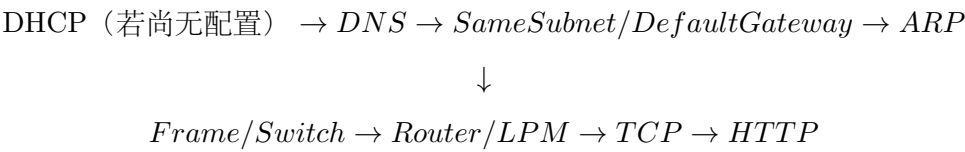
5.11 一个网络请求的一生：总图中的箭头怎样读

第一次访问远程网站时，可用下面链条作为**专题导航器**：



它不是说所有真实网络都只有这七步，而是提示学习时分别去检查：主机网络配置是否已有；域名怎样解析；目的 IP 是同网段还是经网关；下一跳 MAC 如何得到；帧和分组怎样逐跳前进；目的主机怎样交给正确端口/连接；最后 bytes 怎样被 HTTP/DNS 等应用协议解释。

具体第一次访问可近似追：



每一段只负责调用相应专题，绝不在总图里展开协议细节。

5.12 网络最重要的概念边界：区别在哪里，题目怎么用

概念边界	真正区别	题目信号	混淆后的错误
------	------	------	--------

Name $\neq$ IP $\neq$ MAC $\neq$ Port	名字指应用对象；IP 定位网络层目的；MAC 服务当前链路交付；Port 区分主机内端点	DNS、子网/路由、ARP/Ethernet、socket/port 分别对应不同作用域	把“下一跳 MAC”当成最终目标主机 MAC，或认为 Port 用来路由
Routing vs. Forwarding	Routing 生成/更新规则；Forwarding 使用现有规则处理当前 packet	DV / LS / RIP / OSPF / BGP 是 routing；LPM 查表是 forwarding	用 Dijkstra 解释每个 packet 的实时转发过程
Reliability $\neq$ Flow $\neq$ Congestion	分别保护数据语义、接收方、网络路径	seq/ACK；rwnd；cwnd/slow start 是三组信号	看到“窗口变小”就无法判断为什么变
Transmission Delay $\neq$ Propagation Delay	一个取决于 packet 长度和发送速率 L/R ；一个取决于距离和传播速度 D/v	改 packet 大小只影响 transmission；改距离只影响 propagation	时延题把带宽和距离混进同一个公式
Connection $\neq$ Physical Path	TCP connection 是端点维护的逻辑状态；中间真实路径可由多条链路和路由器组成并可能变化	题目问 socket / 4-tuple 是 connection；问 router / link 是 path	认为 TCP 建连时沿途路由器都保存连接状态
Link Reliability $\neq$ End-to-End Reliability	都可用 ACK/重传，但一个修复一跳，一个保证两个端点之间的服务语义	Wi-Fi / 链路重传与 TCP 重传 Scope 不同	因链路可靠就认为端到端绝不会丢
TCP State $\neq$ Router State	seq / window 属端点；prefix / next-hop 属路由器	看状态字段“谁需要读它做决定”	把 TCP ACK 变化用于更新普通路由器转发表
Layering $\neq$ Physical Law	分层是隔离变化的架构/分析方法，不是自然界强制边界	NAT / VPN / Proxy 等现实实现会跨传统层次，但分析仍可分 Scope	认为任何功能只能严格存在于某一层，无法理解跨层接口

5.13 网络做题入口：八个词怎样变成动作

408 训练：网络八步检查链

$Scope \rightarrow Object \rightarrow StateOwner \rightarrow Event$

↓

$Transition \rightarrow Feedback \rightarrow Bottleneck/Queue \rightarrow Target/Cost$

1. **Scope**: 先说这是应用、端到端、跨网路径、一跳还是物理信道的问题。
2. **Object**: 当前处理的是 message、segment、datagram、frame 还是 bit/symbol。
3. **StateOwner**: 做决定所需的表/窗口/序号到底存在哪个端点或设备。
4. **Event**: 收到 packet/ACK/routing update, 还是 timer/collision 等事件?
5. **Transition**: 哪张表、哪个窗口、哪个序号或队列发生改变?
6. **Feedback**: 这个动作会向谁发什么信息, 怎样改变远端下一步判断?
7. **Bottleneck/Queue**: 如果是性能问题, 哪个有限资源真正成为瓶颈, 是否排队?
8. **Target/Cost**: 最后才计算时延、吞吐、利用率、窗口、地址范围等目标量。

小例子：路由器收到一个 IP 分组，问输出接口

Scope=Network/当前路由器的数据平面; Object=IP datagram; StateOwner= 该路由器 forwarding table; Event=packet arrival; Transition/Action= 对 DestinationIP 做 Longest Prefix Match, 得到 NextHop/Interface; 这里通常不需要分析 TCP seq, 也不需要运行 OSPF。若题目进一步问“这张表怎样学出来”，才切换到 Routing 专题。

5.14 一页压缩

闭眼后应该剩下什么

$$\text{Scope} \rightarrow \text{Local State} \xrightarrow{\text{Message/Timer}} \text{Transition} \rightarrow \text{Feedback} \rightarrow \text{Cost}$$

把它翻译成中文就是：这个决定负责多远？谁手里有什么局部信息？收到什么或等了多久？它因此改了什么并做了什么？怎样通过消息影响别的节点？最后付出了什么代价？这才是网络母模型，而不是一串英文标签。

四科接口：真正的系统不是四块并排知识

6.1 数据结构 × 计组：渐近复杂度为什么不是实际机器时间

数组与链表可能都有线性遍历复杂度，但连续内存更容易利用 cache line 和预取；B/B+ 树的存在则直接说明：当随机 I/O 成为主成本时，结构目标从减少比较变成减少层数和 I/O。算法成本必须最终落到底层机器成本模型。

6.2 计组 × OS：机制与策略的软硬件边界

硬件提供特权级、MMU/TLB、trap/interrupt、timer、atomic instruction、DMA 等受控机制；OS 建立页表、调度策略、等待队列、文件和设备抽象，并决定何时使用这些机制。

Hardware provides enforceable mechanisms; OS maintains abstractions, state and policy.

6.3 OS × 网络：packet 最终必须变成进程可见的数据

网络协议不是漂浮在应用之下的抽象云。接收路径最终会经历 NIC 收帧、DMA/内存、内核协议栈、socket buffer、等待进程唤醒与调度，再回到用户态。因此 socket 是网络语义与 OS 资源/执行模型的关键接口。

6.4 一条“应用读取网络数据”的跨科路径

远端 Application → Transport/IP/Link → NIC → DMA/Main Memory → Interrupt/Kernel
→ Network Stack → Socket Buffer → Blocked Process Wakeup → Scheduler → User
read/recv → Application

在这条路径里：

- 网络回答“远端数据怎样抵达本机”；
- 计组回答“CPU、Cache、内存、DMA、中断怎样真实执行”；
- OS 回答“内核怎样组织 buffer、进程等待和控制权”；
- 数据结构回答“队列、散列、树、ring buffer 等状态怎样被高效组织”。

6.5 跨科综合题的统一顺序

408 训练：先分层，再追踪：箭头表示思考顺序

Object → Layer/Scope → State/Representation → Event/Operation

↓

Mapping/DataPath → Queue/Competition → Invariant → Cost

1. **Object**：当前到底在追 packet、instruction、process、page 还是数据结构节点？
2. **Layer/Scope**：它现在处在网络哪一作用域、OS 哪个抽象、计组哪一级状态？先锁定世界。
3. **State/Representation**：当前能直接查到哪些表、寄存器、队列、指针或缓存状态？
4. **Event/Operation**：谁做了什么，导致状态有资格变化？
5. **Mapping/DataPath**：需要经过哪条映射链或数据通路才能到下一层？
6. **Queue/Competition**：有没有等待、冲突、带宽/端口/CPU 竞争？
7. **Invariant**：每次状态变化后什么必须仍然正确？
8. **Cost**：最终把完整路径换算成题目要求的时间、空间、I/O、吞吐或比较次数。

以“应用阻塞读取网络数据”为例：Object 先从 remote packet 转为本机 socket/process；网络 Scope 负责分组抵达，本机 NIC/内存进入计组/OS 接口；State 包括 NIC descriptor、socket buffer 和 blocked process；packet arrival/DMA completion/interrupt 是事件；协议栈和 socket 映射是处理路径；进程等待队列体现 Queue；最终必须保证数据属于正确 socket 且用户缓冲区访问合法；最后才谈 DMA、唤醒、调度和 copy 的性能代价。这样一条综合链真正告诉你“何时切科”，而不是把四科名词堆在一起。

专题所有权与学习路线：总图怎样和后续手册配合

7.1 Canonical Owner 原则

同一知识点只能有一个专题负责完整解释。其他手册只能：

- **Use**：调用结论；
- **Bridge**：解释接口；
- **Integration**：在真实生命周期中串联；

不能重新复制完整机制。

知识	Canonical Owner	典型调用者
Sliding Window / GBN / SR	NW-03 可靠传输	TCP、网络性能综合
Flow Control	NW-06 Transport/TCP	TCP 综合
Congestion Control	NW-07 拥塞	TCP/网络请求综合
ARP	NW-04 IP/Forwarding	NW-02 单跳、网络综合
Page Fault / COW	OS-04 VM	fork/mmap/OS 综合
DMA	OS-05 I/O + CO-07 硬件视角	OS 综合、OS×Net
Open File State	OS-06 File System	fork/read/unlink 综合
Cache Mapping	CO-06 Cache	LOAD、性能综合
Graph Shortest Path	DS-07 Graph Algorithms	NW-05 Routing 的算法基础

7.2 推荐学习方式

对任何专题使用同一节奏：

总图定位 → 专题打穿 → 题目验证 → 桥梁/综合重连 → 回到一页压缩

这条链不是出版流程，而是实际学习流程：

1. **总图定位**：上课/看书前先知道这一章在母模型哪一格，例如“路由”主要深入 Local State + Message + Convergence；
2. **专题打穿**：进入专题手册，把具体对象、算法、状态机、公式和题型学清；
3. **题目验证**：做题时检查自己究竟卡在模型、识别、路径、执行还是检查；
4. **桥梁/综合重连**：学完后把它接回邻近专题，例如 TCP 的可靠性调用 NW-03，而拥塞状态归 NW-07；
5. **一页压缩**：最后重新用自己的话解释母模型，确认具体知识能重新挂回地图。

7.3 什么时候应该修改总手册

只有出现以下变化才改总图：

1. 某门课的母问题/母模型被证明不够准确；
2. 专题边界或 canonical owner 发生变化；
3. 出现能够解释多个专题的新上位关系；
4. 长期做题证据证明某个关键概念边界必须进入学科总图。

单道题的技巧、某个公式细节、某个协议字段、某种算法实现差异不直接进入总手册。

覆盖导航：总图只指出入口，不复制清单

详细考试覆盖清单统一由《408 四科统一方法论手册》维护；总图只保留四科覆盖域，防止同一清单在两本总册中分叉。

学科	覆盖域	检查入口
数据结构	复杂度、线性结构、树、图、查找与索引、串、内部与外部排序	先按核心模型定位 Relation、Operations、Representation、Invariant 与 Cost，再到四科方法论核对详细条目
计算机组成原理	数据表示、ISA、存储层次、数据通路与控制、流水线、总线与 I/O、性能	先定位软件可见语义、数据位置、时序与提交，再核对机制覆盖
操作系统	控制权、进程与调度、并发、虚拟内存、I/O、文件系统与持久化	先定位对象、关系、队列与事件，再进入对应 OS Topic
计算机网络	分层与性能、物理/链路、可靠传输、IP 与路由、TCP/UDP、应用层	先定位作用域和状态所有者，再进入对应网络专题

总图 / 专题边界：覆盖层的停止条件

总图确认“专题入口存在”后即停止；具体公式、字段、算法步骤和题型清单只在四科方法论或专题手册维护。这样总图负责导航，深读手册负责解释，覆盖清单负责防漏。

最后压缩：考前真正应该留下什么

四科一句话

- **数据结构**：为了操作需求，选择并维护合适的信息表示；
- **计组**：把 ISA 的软件语义实现成真实硬件的数据流与时间过程；
- **OS**：在硬件机制上维护受保护的**对象、关系、队列**和资源状态；
- **网络**：自治节点只凭局部状态和消息反馈共同建立端到端通信。

四条母链

DS: $Problem \rightarrow Relation \rightarrow Operations \rightarrow Representation \rightarrow Invariant \rightarrow Cost$

CO: $ISAState \rightarrow Data \rightarrow Path/Control \rightarrow Timing \rightarrow Commit \rightarrow Cost$

OS: $Objects/Relations/Queues \xrightarrow{Event+Mechanism+Policy} NewState \mid Invariant, Cost$

NET: $Scope \rightarrow LocalState \xrightarrow{Message/Timer} Transition \rightarrow Feedback \rightarrow Cost$

最终学习原则

不要把“知道名词”误认为“拥有模型”，也不要“会背母链”误认为“会用母模型”。真正掌握一个知识点，应当能够：

1. 用自己的中文解释母模型每一格是什么意思；
2. 指出它在教材/专题中的具体章节实例；
3. 给一个小题，按箭头顺序真正走一遍；
4. 遇到 $A \neq B$ 时说出判据、题目信号和混淆后果；
5. 最终回答它解决什么问题、依赖什么状态/表示、由什么操作或事件触发、维护什么不变量、代价从哪里来、应该挂到哪个专题。

总图负责让你永远知道自己站在哪里；专题负责让你知道这一块为什么这样运行；题目负责证明这个模型是否真的属于你。